

IIT Jammu Summer School 2026

IoT Assignment 3

Name: Swarnava Dey

Roll no. : IoT050

Course: IoT and Drones

Submitted To: Mr. Majid Majeed

Github Repo: <https://github.com/Swarnava24/IoT-Summer-School-2026.git>

Q36. MQTT vs HTTP Comparison

Ans36:

MQTT vs HTTP Comparison

Feature	MQTT	HTTP
Architecture Pattern	Publish/Subscribe through an MQTT broker	Client/Server (Request–Response)
Data Transfer Model	Asynchronous; devices publish to topics and subscribers receive updates	Synchronous; client sends a request and waits for the server's response
Power Consumption	Very low due to lightweight packets and persistent connections	Higher because each request requires additional protocol overhead
Latency	Low latency; suitable for real-time communication	Higher latency due to repeated request-response cycles
Use Case Suitability for IoT	Excellent for IoT applications such as smart homes, industrial monitoring, agriculture, and sensor networks	Better suited for web services, REST APIs, dashboards, and occasional data retrieval
Security Considerations	Supports username/password authentication, TLS/SSL encryption, and access control through brokers	Supports HTTPS (TLS/SSL), authentication tokens, API keys, OAuth, and other web security mechanisms

For a **1000-node smart agriculture deployment**, **MQTT** is the preferred communication protocol.

Reasons:

1. **Low Bandwidth Usage:** MQTT uses a lightweight messaging format, reducing network traffic even with thousands of sensor nodes.
2. **Low Power Consumption:** Agricultural sensors are often battery- or solar-powered. MQTT minimizes communication overhead, helping devices operate longer between charges.
3. **Low Latency:** Sensor readings such as soil moisture, temperature, and humidity can be delivered quickly to the central monitoring system.
4. **Scalability:** A single MQTT broker can efficiently manage communication with thousands of connected devices using topics.
5. **Reliable Communication:** MQTT supports different Quality of Service (QoS) levels (0, 1, and 2), allowing developers to choose the appropriate balance between reliability and network overhead.
6. **Efficient Data Distribution:** Multiple applications (e.g., irrigation control, weather dashboard, and mobile app) can subscribe to the same sensor topics without increasing the workload on sensor nodes.
7. **Reduced Network Traffic:** Devices only publish data when necessary, and subscribers receive updates automatically, eliminating the need for continuous polling as required by HTTP.

Q37. Wi-Fi Security for IoT

Ans37:

Wi-Fi Security for IoT

Wi-Fi-enabled IoT devices such as the ESP32 are vulnerable to several wireless attacks if not properly secured. One common attack vector is exploiting weaknesses in **WPA2**, particularly through weak passwords or misconfigured networks. Attackers may also use an **evil twin attack**, where they create a fake Wi-Fi access point with the same network name (SSID) as a trusted network. If an IoT device connects to the fake access point, attackers can intercept data or steal login credentials.

To improve the security of ESP32-based IoT systems, developers should follow several best practices. Using **WPA3** instead of WPA2 provides stronger encryption and protection against password guessing attacks. When communicating with cloud services, **HTTPS with certificate validation or certificate pinning** should be used to verify the server's identity and prevent man-in-the-middle attacks. Regular firmware updates, strong unique passwords, and disabling unused network services further enhance device security.

Hardcoding Wi-Fi passwords or API keys directly into the source code is dangerous because anyone with access to the code or firmware can retrieve these credentials. If the code is uploaded to a public GitHub repository, attackers can easily obtain sensitive information and gain unauthorized access to the network. A safer approach is to store credentials in a separate configuration file (such as config.h) that is excluded from version control using .gitignore, or use secure storage mechanisms provided by the device.

References:

1. Wi-Fi Alliance. *WPA3™ Security*. <https://www.wi-fi.org/discover-wi-fi/security>
2. Open Worldwide Application Security Project (OWASP). *OWASP Internet of Things Project*. <https://owasp.org/www-project-internet-of-things/>

Q38. LoRa & LPWAN – Long Range IoT

Ans 38:

LoRa & LPWAN – Long Range IoT

LPWAN (Low Power Wide Area Network) is a wireless communication technology designed for IoT devices that need to send small amounts of data over long distances while consuming very little power. It is commonly used in agriculture, smart cities, and environmental monitoring.

LoRa (Long Range) achieves long-range communication by using a **Chirp Spread Spectrum (CSS)** modulation technique. It also uses a **Spreading Factor (SF)**, where a higher SF increases communication range and reliability but lowers data rate. This allows battery-powered devices to transmit data over several kilometers while using minimal energy.

Feature	LoRa	Zigbee	Z-Wave
Range	2–15 km	10–100 m	30–100 m
Power Consumption	Very Low	Low	Low
Data Rate	Low	High	Medium
Best Use	Smart agriculture, smart cities	Home automation	Home automation

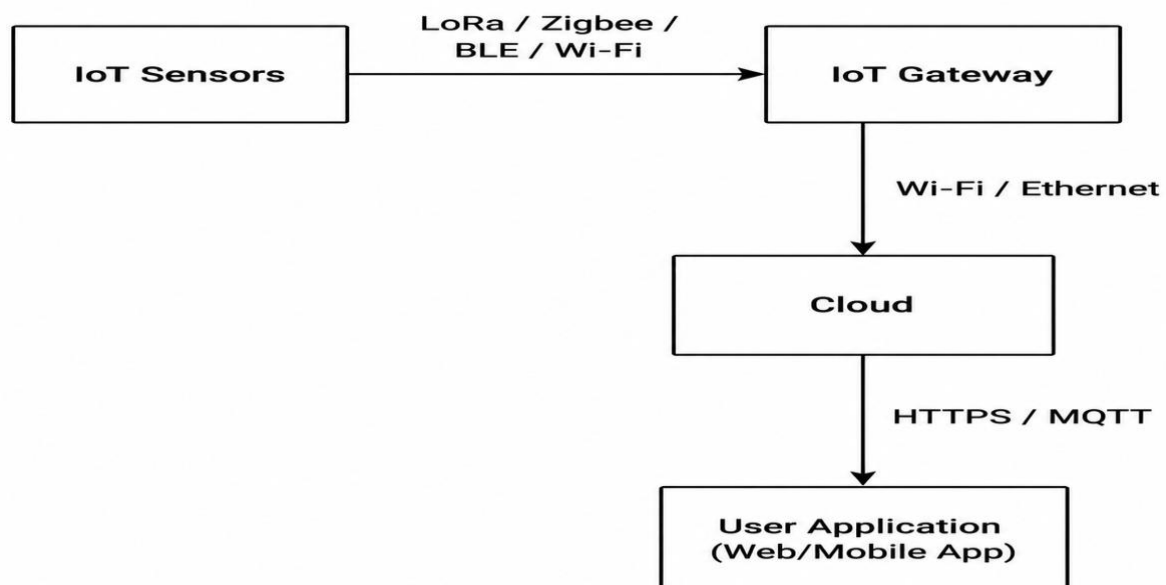
A real-world example in India is the **Smart City Mission**, where LoRa-based sensor networks are deployed in cities such as **Pune** and **New Delhi** for applications like smart street lighting, water metering, waste management, and environmental monitoring. LoRa's long range, low power consumption, and ability to connect thousands of devices make it an ideal choice for large-scale IoT deployments.

Q39. What is an IoT gateway? Draw a simple diagram showing the relationship between: IoT Sensor Nodes → Gateway → Cloud → User Application. What protocols are typically used at each layer?

Ans 39:

IoT Gateway

An **IoT gateway** is a device that connects IoT sensor nodes to the internet or cloud. It collects data from sensors, converts different communication protocols if needed, performs basic processing, and securely forwards the data to cloud services. The gateway also sends commands from the cloud back to the IoT devices.



Protocols Used

Layer	Common Protocols
IoT Sensor → Gateway	LoRa, Zigbee, BLE, Wi-Fi
Gateway → Cloud	MQTT, HTTP/HTTPS, CoAP
Cloud → User Application	HTTPS, MQTT, WebSockets

Q40. Explain QoS (Quality of Service) levels in MQTT: QoS 0 (At most once), QoS 1 (At least once), QoS 2 (Exactly once). In an IoT water level monitoring system, which QoS level would you choose and why?

Ans 40:

MQTT QoS (Quality of Service) Levels

QoS Level	Description	Reliability
QoS 0 – At most once	Message is sent only once without confirmation. It may be lost if the network fails.	Lowest
QoS 1 – At least once	Message is delivered at least once. The receiver acknowledges it, but duplicate messages may occur.	Medium
QoS 2 – Exactly once	Message is delivered exactly once using a four-step handshake. No duplicates occur.	Highest

Which QoS would I choose for an IoT Water Level Monitoring System?

I would choose **QoS 1 (At least once)** because it provides reliable message delivery while keeping communication overhead low. In a water level monitoring system, receiving duplicate readings occasionally is acceptable, but losing critical water level data could lead to incorrect monitoring or delayed alerts. QoS 2 offers the highest reliability but requires more bandwidth and processing, making it less efficient for most IoT applications. Therefore, **QoS 1 provides the best balance between reliability and performance.**

Q41. Smart Home Air Quality Monitor

A family in a Milan(Europe) home is concerned about indoor air quality during winter when they burn wood for heating. Design an IoT-based Air Quality Monitoring System that continuously measures CO/smoke levels (MQ-2 sensor), temperature and humidity (DHT11), and alerts residents when air quality drops below safe thresholds. The system must work without the internet when the router is down.

Ans41:

Github Link: https://github.com/Swarnava24/IoT-Summer-School-2026/tree/main/Assignment/IoT_Assignment3/QUES41

Q42. Automated Plant Watering System

A J&K horticulture farmer wants to automate irrigation for a small saffron farm plot. The system should read soil moisture every 30 minutes and activate a water pump only when the soil is dry. Saffron is sensitive to overwatering — the system must also prevent watering if it rained recently (use LDR to detect low light/overcast sky as a rain proxy and also leverage the use of api also). The farmer must be able to manually override via a button.

Ans42:

Github Link: https://github.com/Swarnava24/IoT-Summer-School-2026/tree/main/Assignment/IoT_Assignment3/QUES42

Q43. Smart Door Lock with OTP

A university hostel wants to replace physical keys with a PIN-based entry system that also generates a One-Time Password. Design a system where a student enters their 4-digit ID on a keypad, the system generates a 6-digit OTP, sends it over Bluetooth to their phone, and only grants access if they enter the OTP within 30 seconds.

Ans43:

Github Link: https://github.com/Swarnava24/IoT-Summer-School-2026/tree/main/Assignment/IoT_Assignment3/QUES43

Q44. PIR-Based Security Camera Trigger

A small shop in Haryana, Rohtak wants a low-cost security alert system. When motion is detected after shop hours (manually configured time window), the system should sound an alarm, flash lights, and optionally send a notification. Build the motion-detection and alerting module.

Ans 44:

Github Link: https://github.com/Swarnava24/IoT-Summer-School-2026/tree/main/Assignment/IoT_Assignment3/QUES44

Q45. Smart Greenhouse Controller

Floriculture farmers in Kashmir growing tulips and roses need automated greenhouse climate control. Design a multi-parameter monitoring and control system that maintains optimal growing conditions: Temperature 18–22°C, Humidity 60–70%, and sufficient light for 12 hours per day.

Ans 45:

Github Link: https://github.com/Swarnava24/IoT-Summer-School-2026/tree/main/Assignment/IoT_Assignment3/QUES45

Q46. River Water Level Alert (Flood Detection)

Given the frequent flooding near the Tawi and Chenab rivers, design a low-cost IoT flood-early-warning sensor node using an HC-SR04 ultrasonic sensor to monitor water level in a model river (a glass tank). Trigger alerts at predefined levels and transmit data wirelessly.

Ans 46:

Github Link: https://github.com/Swarnava24/IoT-Summer-School-2026/tree/main/Assignment/IoT_Assignment3/QUES46

Q47. Heart Rate & SpO2 Monitor (Simulation)

Using a MAX30102 pulse oximeter module (or simulate with an LDR as a proxy sensor), build a health monitoring prototype that measures pulse rate and blood oxygen saturation, displays values, and alerts on abnormal readings. This is a simulation for learning sensor interfacing, not a medical device.

Ans 47:

Github Link: https://github.com/Swarnava24/IoT-Summer-School-2026/tree/main/Assignment/IoT_Assignment3/QUES47

Q48. Design Challenge: IoT for Jammu Smart City

J&K Government is planning a Smart City project for Jammu. As an IoT engineer, propose a system to monitor and manage 3 of the following (choose any 3): Street lighting, Traffic flow, Parking availability, Air quality, Water supply pressure, Waste bin fill levels. For each:

- Identify the sensor(s) needed
- Identify the microcontroller / connectivity module
- Describe the data flow: Sensor → Protocol → Cloud → Dashboard
- Estimate approximate cost per node (in INR)

Ans 48:

Github Link: https://github.com/Swarnava24/IoT-Summer-School-2026/tree/main/Assignment/IoT_Assignment3/QUES48

Q49. Security Vulnerability Analysis

You are given a poorly written IoT code snippet that connects an ESP8266 to a public Wi-Fi network. The code hardcodes the SSID and password, uses unencrypted HTTP (not HTTPS), publishes sensor data to a public MQTT broker without authentication, and never validates incoming commands. Identify at least 5 security vulnerabilities, explain the risk of each, and provide corrected code for at least 2 vulnerabilities.

Ans 49:

Github Link: https://github.com/Swarnava24/IoT-Summer-School-2026/tree/main/Assignment/IoT_Assignment3/QUES49

Q50. Reflection & Learning Portfolio

Describe what you built and what you learned. How will you apply IoT skills in your field of study or future career? What would you build next — describe your own IoT project idea with sensors, connectivity, and application. (Minimum 400 words, personal and reflective)

Ans 50:

Reflection on the IIT Jammu Summer School IoT Program

The IIT Jammu Summer School on the Internet of Things (IoT) provided me with a practical understanding of how embedded systems, sensors, networking, and cloud technologies come together to build real-world smart applications. Throughout the program, I worked on a wide variety of projects that gradually introduced me to different aspects of IoT development, starting from basic microcontroller programming and progressing to complete Wi-Fi-enabled systems capable of communicating with cloud platforms.

During the course, I built several projects including traffic light controllers, reaction time games, digital piano systems, environmental monitoring stations, smart street lights, password-protected access systems, vending machine state machines, ultrasonic parking sensors, relay-controlled automation systems, patient health monitoring, Wi-Fi web servers, MQTT-based sensor publishers, cloud-connected water tank monitoring systems, automated plant watering systems, smart greenhouse controllers, and an air quality monitoring system using ESP32. These projects helped me understand how different sensors such as

DHT22, MQ-2, ultrasonic sensors, LDRs, soil moisture sensors, PIR sensors, and keypads can be integrated with microcontrollers to create intelligent systems.

One of the most valuable lessons I learned was that IoT is much more than connecting a sensor to the internet. A reliable IoT system requires proper sensor calibration, filtering noisy sensor readings, implementing control algorithms, handling communication failures, designing user interfaces, and ensuring that the system continues to operate safely even when internet connectivity is unavailable. I also learned the importance of communication protocols such as HTTP and MQTT, and when each protocol is most suitable depending on latency, bandwidth, and reliability requirements.

The course also strengthened my programming skills in embedded C++ using the Arduino framework. I became comfortable working with ESP32 and Arduino boards, using libraries for displays, sensors, Wi-Fi communication, Bluetooth, and cloud services such as ThingSpeak and OpenWeatherMap. I gained experience with Wokwi for rapid prototyping before deploying projects on real hardware. Learning how to debug hardware-software interactions and organize projects into reusable modules has improved my confidence in developing embedded systems.

As a Computer Science student, I believe these IoT skills complement my software development background exceptionally well. Modern applications increasingly involve physical devices that generate real-time data. Understanding embedded programming, wireless communication, and cloud integration enables me to develop complete end-to-end solutions instead of only software applications. In my future career, I hope to work in areas such as embedded systems, artificial intelligence, robotics, or smart infrastructure, where IoT plays a critical role. The experience gained during this summer school has provided me with a strong foundation for pursuing these fields.

If I were to build my own IoT project next, I would develop an AI-powered Smart Campus Energy Management System. The system would use occupancy sensors (PIR), LDRs, temperature and humidity sensors, and smart energy meters installed across classrooms and laboratories. ESP32 nodes would communicate using MQTT over Wi-Fi with a central server that collects and analyzes the data. Machine learning models would predict occupancy patterns and automatically control lighting, fans, and air conditioning to reduce unnecessary energy consumption while maintaining user comfort. A web dashboard and mobile

application would provide live monitoring, historical analytics, energy usage reports, and maintenance alerts. The system could also notify facility managers about equipment failures or abnormal power consumption, making campus operations more efficient and sustainable.

Overall, this summer school has transformed my understanding of IoT from a theoretical concept into practical engineering experience. The hands-on projects taught me how to design, build, debug, and deploy connected embedded systems. More importantly, they showed me how IoT technologies can be applied to solve real-world problems in healthcare, agriculture, smart homes, industrial automation, and environmental monitoring. I am excited to continue learning in this domain and to combine IoT with artificial intelligence, cloud computing, and data analytics to build innovative solutions in the future.